

如何在Github上提交PR (pull request)

- 如何提交代码
- 合入不规范
 - 提交信息不规范
 - Markdown 代码格式
 - pull request里的commit数量
 - 代码注释
 - 说明具体是哪种方法
 - 代码规范
 - 代码逻辑
 - 处理冲突

以下在 <https://github.com/youngyangyang04/leetcode-master> 上提交pr为为例

如何合入代码

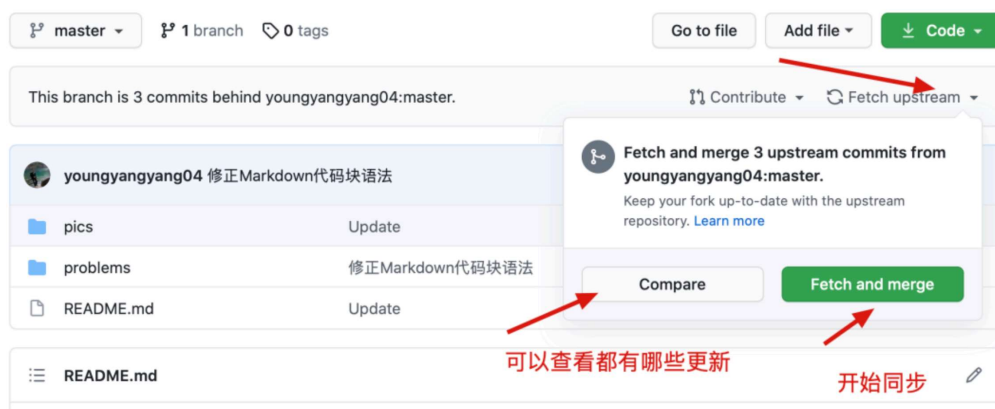
首先来说一说如何合入代码，不少录友还不太会使用github，所以这里也做一下科普。

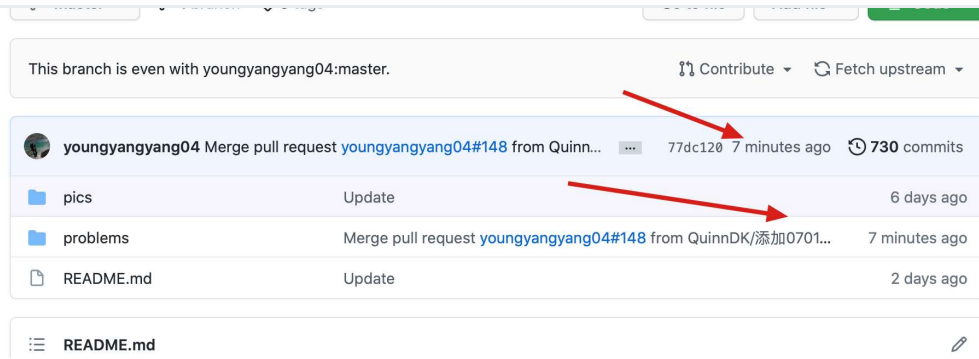
我特意申请一个新的Github账号，给大家做一个示范。

需要强调一下，一个commit就只更新一道题目，不要很多题目一起放在一个commit里，那样就很乱。

首先LeetCode-Master每天都有更新，如何保持你fork到自己的仓库是最新的版本呢。

点击[这里](#)Fetch upstream。

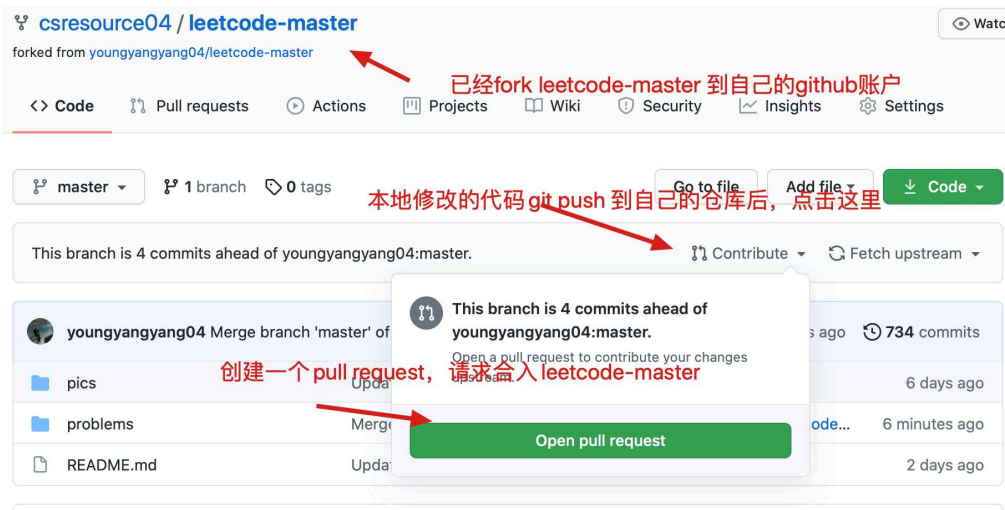




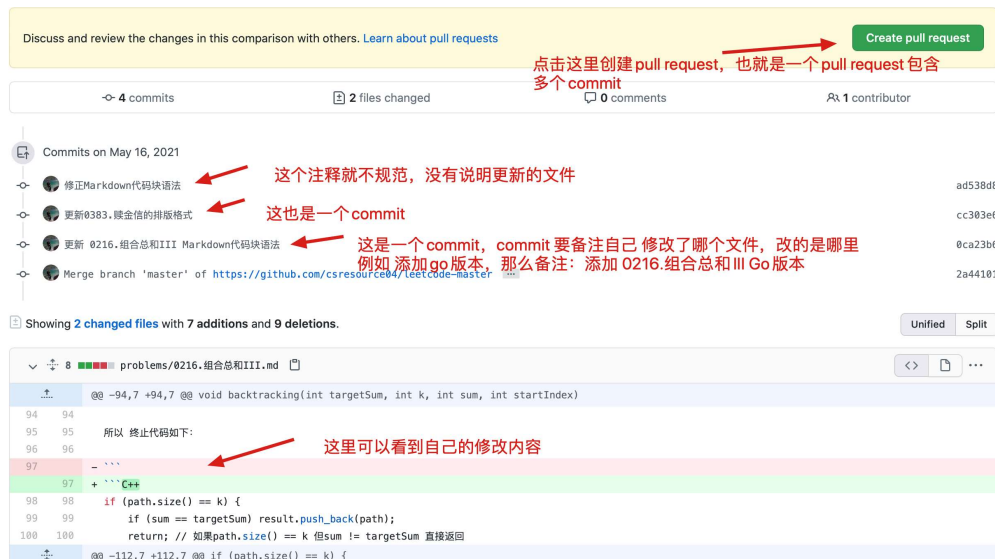
注意这时是你的远端仓库为最新版本，本地还不是最新的，本地要git pull一下。

基于最新的版本，大家在去提交代码。

如何提交代码呢，首先把自己的代码提交到自己的fork的远端仓库中，然后open pull request，如图：



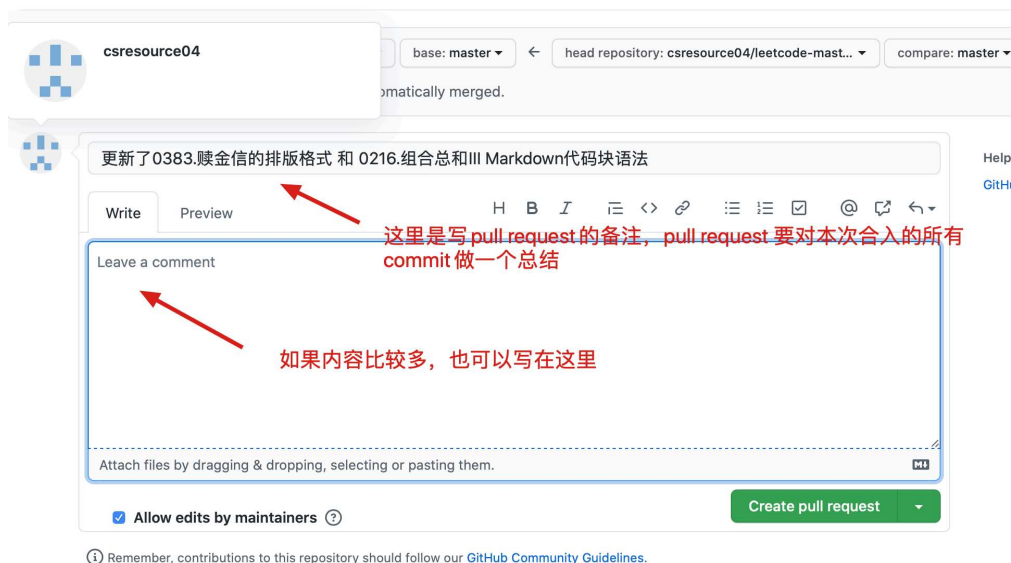
点击 open pull request之后，就是如下画面，一个pull request有多个commit。



可以了

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).



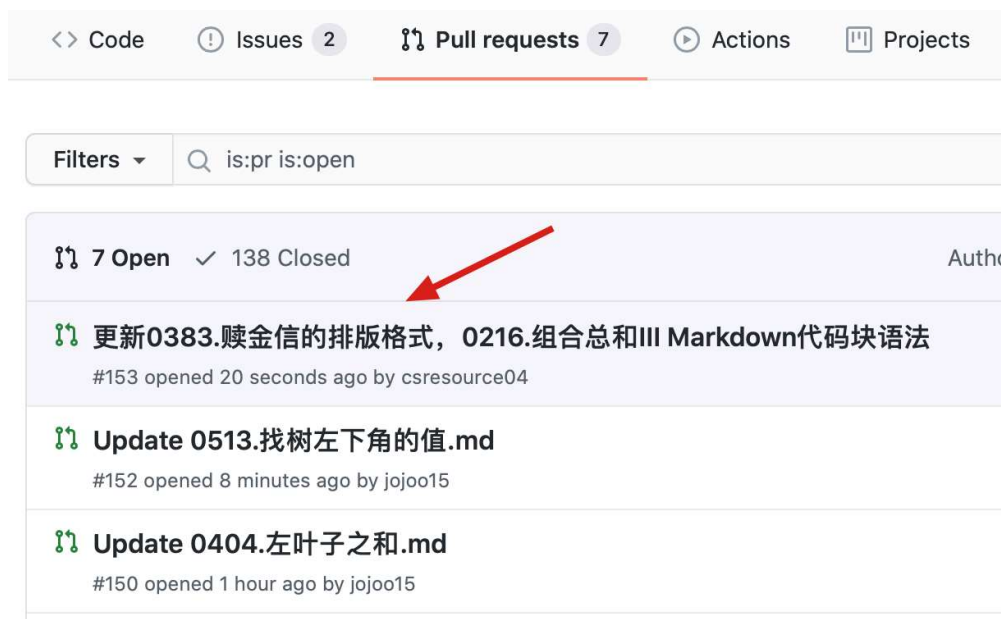
更新了0383.赎金信的排版格式 和 0216.组合总和III Markdown代码块语法

这里是写 pull request 的备注, pull request 要对本次合入的所有 commit 做一个总结

如果内容比较多, 也可以写在这里

Create pull request

此时你就提交成功了, 我会在项目中的pull requests 处理列表里看到你的请求。



Code Issues 2 Pull requests 7 Actions Projects

Filters is:pr is:open

7 Open 138 Closed

更新0383.赎金信的排版格式, 0216.组合总和III Markdown代码块语法
#153 opened 20 seconds ago by csresource04

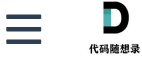
Update 0513.找树左下角的值.md
#152 opened 8 minutes ago by jojoo15

Update 0404.左叶子之和.md
#150 opened 1 hour ago by jojoo15

然后如果你发现自己的代码没有合入多半是有问题, 如果有问题都会有会在pull request里给出留言的,

注意事项

提交信息不规范



commit 说清楚本文件多了哪些修改，而pull request则是对本次合入的所有commit做一个总结性描述。

commit备注，举例：添加Rust python3，那么commit备注就是：添加0001两数之和 Rust python3 版本

而pull request 如果只有一个commit，那么就也是：添加0001两数之和 Rust python3 版本

如果是多个commit，则把本次commit都描述一遍。

Markdown 语法

关于 Markdown 代码格式，例如 添加C++代码，需要有代码块语法

```C++ C++代码 ```

例如这个commit，在添加java代码的时候，就直接添加代码

```
3 problems/0121.买卖股票的最佳时机.md [icon]
@@ -198,7 +198,18 @@ public:
8
9
0 Java:
-
1 + public int maxProfit(int[] prices) {
2 + int minPrice = Integer.MAX_VALUE;
3 + int profit = 0;
4 + for (int i = 0; i < prices.length; i++) {
5 + if (minPrice > prices[i]) {
6 + minPrice = prices[i];
7 + } else if (profit < prices[i] - minPrice){
8 + profit = prices[i] - minPrice;
9 + }
0 + }
1 + return profit;
2 + }
3
```

正确的格式应该是这样：

```
+ class Solution {
+ public void reverseString(char[] s) {
+ int l = 0;
+ int r = s.length - 1;
+ while (l < r) {
+ s[l] ^= s[r]; //构造 a ^ b 的结果, 并放在 a 中
+ s[r] ^= s[l]; //将 a ^ b 这一结果再 ^ b, 存入b中, 此时 b = a, a = a ^ b
+ s[l] ^= s[r]; //a ^ b 的结果再 ^ a, 存入 a 中, 此时 b = a, a = b 完成交换
+ l++;
+ r--;
+ }
+ }
+ }
+ }
+ }
```

一般发现问题, 我也会在代码中给出评论:

106106Python:

107-

 **youngyangyang04** Owner Pending

Markdown文件 添加python代码应该这样

``` python  
python代码
```

 Reply...

107+ class Solution(object):

108+ def reverseStr(self, s, k):

109+ """

这样大家也可以学习一些 提交代码的规范方面的知识

有的录友 是添加的代码块语法, 但没有标记是哪种语言, 这样的话 代码就不会针对某种语言高亮显示了, 也比较影响阅读, 例如:

```
96 + ...
97 + type TreeNode struct {
98 + Val int
99 + Left *TreeNode
00 + Right *TreeNode
01 + }
02 + ...
```

应该是 ``go

提交python代码的话，要注释好，是python2还是python3

例如这样：

## 添加0035.搜索插入位置Python3版本 #69

 Open tianshengsui wants to merge 1 commit into youngyangyang:master from tianshengsui:master

Conversation 0 Commits 1 Checks 0 Files changed 1

Changes from all commits File filter Jump to

添加0035.搜索插入位置Python3版本

tianshengsui committed 4 hours ago Verified

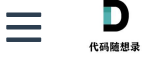
problems/0035.搜索插入位置.md

```
@@ -237,6 +237,22 @@ class Solution {
237 237
238 238
239 239 Python:
240 + ``python3
241 + class Solution:
242 + def searchInsert(self, nums: List[int], target: int) -> int:
243 + left, right = 0, len(nums) - 1
```

当然python2的话，只这么写就行

``python python代码``

pull request里的commit数量



有的求友是一个pull request 包含多个commit。

- ☐  **添加 0090.子集II.md Java版本**  
#48 opened 27 minutes ago by YiwenXie
- ☐  **添加 0078.子集.md Java版本**  
#47 opened 30 minutes ago by YiwenXie
- ☐  **添加 0077.组合优化.md Java版本**  
#46 opened 33 minutes ago by YiwenXie
- ☐  **添加 0077.组合.md Java版本**  
#45 opened 35 minutes ago by YiwenXie
- ☐  **添加 0053.最大子序和.md Java版本**  
#44 opened 38 minutes ago by YiwenXie
- ☐  **添加 0037.解数独.md Java 版本**  
#43 opened 40 minutes ago by YiwenXie
- ☐  **Update 0046.全排列.md**  
#42 opened 1 hour ago by YiwenXie

其实如果大家是平时一天写了两三道题目的话，那么分三个commit，一个pull request提交上来就行。

一个pull request 一个commit也可以，这样大家就会麻烦一点。

但注意一个pull request也不要放太多的commit，一旦有一道题目代码不合格，我没有合入，就这个pull request里影响其他所有代码的合入了。

## 代码注释

提交的代码最好要有注释，这样也方便读者理解。

例如这位录友，在提交Java代码的时候，按照题解的意思对Java版本的代码进行的注释，这就很棒👍



```
problems/0045.跳跃游戏II.md
@@ -143,7 +143,36 @@ public:
143 143
144 144
145 145 + Java:
146 -
146 + ```Java
147 + class Solution {
148 + public int jump(int[] nums) {
149 + if (nums == null || nums.length == 0 || nums.length == 1) {
150 + return 0;
151 + }
152 + //记录跳跃的次数
153 + int count=0;
154 + //当前的覆盖最大区域
155 + int curDistance = 0;
156 + //最大的覆盖区域
157 + int maxDistance = 0;
158 + for (int i = 0; i < nums.length; i++) {
159 + //在可覆盖区域内更新最大的覆盖区域
160 + maxDistance = Math.max(maxDistance,i+nums[i]);
161 + //说明当前一步，再跳一步就到达了末尾
162 + if (maxDistance>=nums.length-1){
163 + count++;
164 + break;
165 + }
166 + //走到当前覆盖的最大区域时，更新下一步可达的最大区域
```



```

@@ -101,7 +101,36 @@ public:
101 101
102 102
103 103 Java:
104 104 -
104 104 + ```Java
105 105 + class Solution {
106 106 + public String reverseStr(String s, int k) {
107 107 + StringBuffer res = new StringBuffer();
108 108 +
109 109 + for (int i = 0; i < s.length(); i += (2 * k)) {
110 110 + StringBuffer temp = new StringBuffer();
111 111 + // 剩余字符大于 k 个，每隔 2k 个字符的前 k 个字符进行反转
112 112 + if (i + k <= s.length()) {
113 113 + // 反转前 k 个字符
114 114 + temp.append(s.substring(i, i + k));
115 115 + res.append(temp.reverse());
116 116 +
117 117 + // 反转完前 k 个字符之后，如果紧接着还有 k 个字符，则直接加入这 k 个字符
118 118 + if (i + 2 * k <= s.length()) {
119 119 + res.append(s.substring(i + k, i + 2 * k));
120 120 + // 不足 k 个字符，则直接加入剩下所有字符
121 121 + } else {
122 122 + res.append(s.substring(i + k, s.length()));
123 123 + }
124 124 + continue;
125 125 + }
126 126 + // 剩余字符少于 k 个，则将剩余字符全部反转。
127 127 + temp.append(s.substring(i, s.length()));
128 128 + res.append(temp.reverse());
129 129 + }
130 130 + return res.toString();

```

详细的代码注释

当然如果大家感觉 已有的代码 不符合以上要求的话，例如 代码思路不够清晰不够规范，注释不够友好，依然欢迎提交优化代码，要记得详细注释哦。

### 为 0122买卖股票的最佳时机II Java代码添加注释

Frank-Tian committed 13 hours ago

```

23 problems/0122.买卖股票的最佳时机II (动态规划) .md
@@ -136,24 +136,31 @@ Java:
136 136 ```java
137 137 // 动态规划
138 138 class Solution
139 139 + // 实现1: 二维数组存储
140 140 + // 可以将每天持有与否的情况分别用 dp[i][0] 和 dp[i][1] 来进行存储
141 141 + // 时间复杂度: O(n), 空间复杂度O(n)
139 142 public int maxProfit(int[] prices) {
140 143 int n = prices.length;
141 144 int[][] dp = new int[n][2];
142 145 dp[0][0] = 0;
144 146 int[][] dp = new int[n][2]; // 创建二维数组存储状态
145 147 dp[0][0] = 0; // 初始状态
143 146 dp[0][1] = -prices[0];
144 147 for (int i = 1; i < n; ++i) {

```

说明具体是哪种方法

下面这位录友做的就很好

```
141 + ```java
142 + /**
143 + * 添加虚节点方式 
144 + * 时间复杂度 O(n)
145 + * 空间复杂度 O(1)
146 + * @param head
147 + * @param val
148 + * @return
149 + */
150 + public ListNode removeElements(ListNode head, int val) {
151 + if (head == null) {
152 + return head;
153 + }
154 + // 因为删除可能涉及到头节点，所以设置dummy节点，统一操作
155 + + ListNode dummy = new ListNode(-1, head);
156 + ListNode pre = dummy;
157 + ListNode cur = head;
158 + while (cur != null) {
159 + if (cur.val == val) {
160 + pre.next = cur.next;
161 + } else {
162 + pre = cur;
163 + }
164 + cur = cur.next;
165 + }
166 + return dummy.next;
167 + }
168 + /**
169 + * 不添加虚拟节点方式 
170 + * 时间复杂度 O(n)
171 + * 空间复杂度 O(1)
172 + * @param head
173 + * @param val
```

```
↑ ... @@ -195,9 +195,51 @@ public:
195 195
196 196 Java:
197 197
198 198 -
199 198 Python:
200 199
200 + 递归法:
201 +
202 + ```python
203 + class Solution:
204 + def minDepth(self, root: TreeNode) -> int:
205 + if not root:
206 + return 0
207 + if not root.left and not root.right:
208 + return 1
209 +
210 + min_depth = 10**9
211 + if root.left:
212 + min_depth = min(self.minDepth(root.left), min_depth) # 获得左子树的最小高度
213 + if root.right:
214 + min_depth = min(self.minDepth(root.right), min_depth) # 获得右子树的最小高度
215 + return min_depth + 1
216 + ```
217 +
218 + 迭代法:
219 +
220 + ```python
221 + class Solution:
222 + def minDepth(self, root: TreeNode) -> int:
223 + if not root:
224 + return 0
```

标明是哪种方法的版本，棒

有的题解，是一起给出了多道题目的讲解，例如项目中0102.二叉树的层序遍历.md 中有八道题目，那么大家添加代码的时候 应该在代码注释上，或者 直接写上 是哪个题目的代码。

## 代码规范

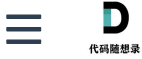
大家提交代码要规范，当然代码可以在力扣上运行通过是最基本的。

虽然我主张没有绝对正确的代码风格，但既然是给LeetCode-Master提交代码，尽量遵循Google编程规范。

经常看我的代码的录友应该都知道，我的代码风格严格按照 Google C++ 编程规范来的，这样看上去会比较整洁。

大家提交代码的时候遇到规范性问题，例如哪里应该有空格，哪里没有空格，可以参考我的代码来。

有一位录友在提交代码的时候会把之前的代码 做一下规范性的调整，这就很棒。



```
434 - if(node==null) return;
434 + //DFS--递归方式
435 + public void checkFun01(TreeNode node, Integer deep){
436 + if (node == null) return;
435 437 deep++;
436 438
```

**代码规范从你我做起！**

## 代码逻辑

**提交的代码要按照题解思路来写。**

虽然大家自己发挥想象空间是好的，但是题解还是要一脉相承，读者看完题解，发现代码和题解不是一个思路的话，那和重新读代码有啥区别了。

所以和题解不是一个思路的代码，除非详细注释了自己的思路 或者 写一段自己代码的描述说明思路和优化的地方，否则我就不会通过合入了哈。

大家的代码 最好也将关键地方放上注释，这样有助于别人快速理解你的代码。

## 处理冲突

在合入的过程中还要处理冲突的代码，理解大家代码的思路，解决冲突，然后在力扣提交一下，确保是没问题。

例如同一道题目，一位录友提交了，我还没处理如何，另一位录友也对这道题也提交了代码，这样就会发生冲突

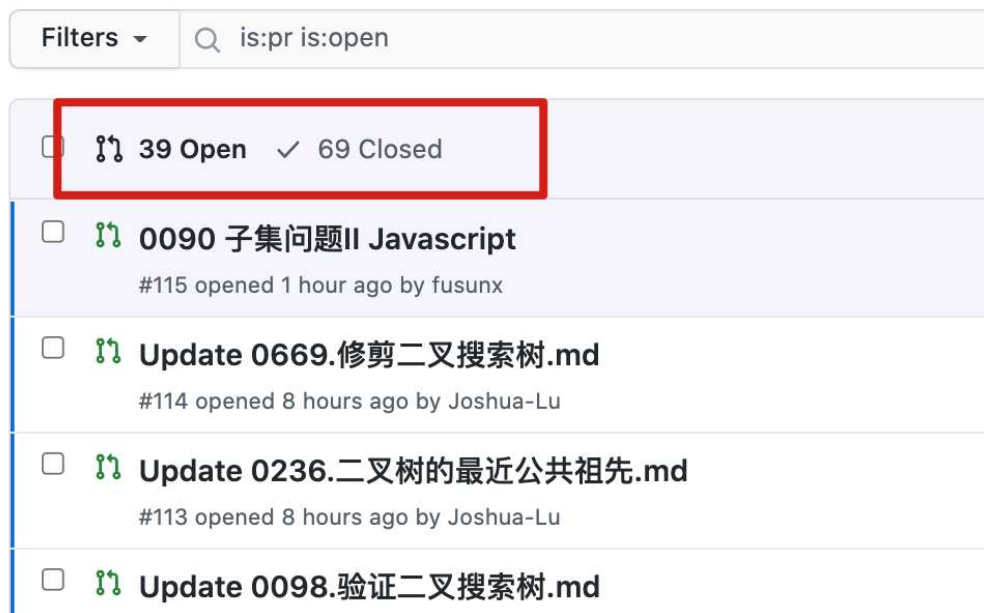


```

154 else return searchBST(root.right, val);
155 }
156 }
157 ***
158
159 迭代法:
160
161 ```java
162 class Solution {
163 public TreeNode searchBST(TreeNode root, int val) {
164 while (root != null)
165 if (val < root.val) root = root.left;
166 else if (val > root.val) root = root.right;
167 else return root;
168 return root;
169 }
170 =====
171 if (root.val > val) return searchBST(root.left, val);
172 return searchBST(root.right, val);
173 }
174 >>>>>> master
175 }

```

大家提交代码的热情太高了，我有时候根本处理不过来，但我必须当天处理完，否则第二天代码冲突会越来越多。



一天晚分别有两位录友提交了 30多道 java代码，全部冲突，解决冲突处理的我脖子疼[哭]

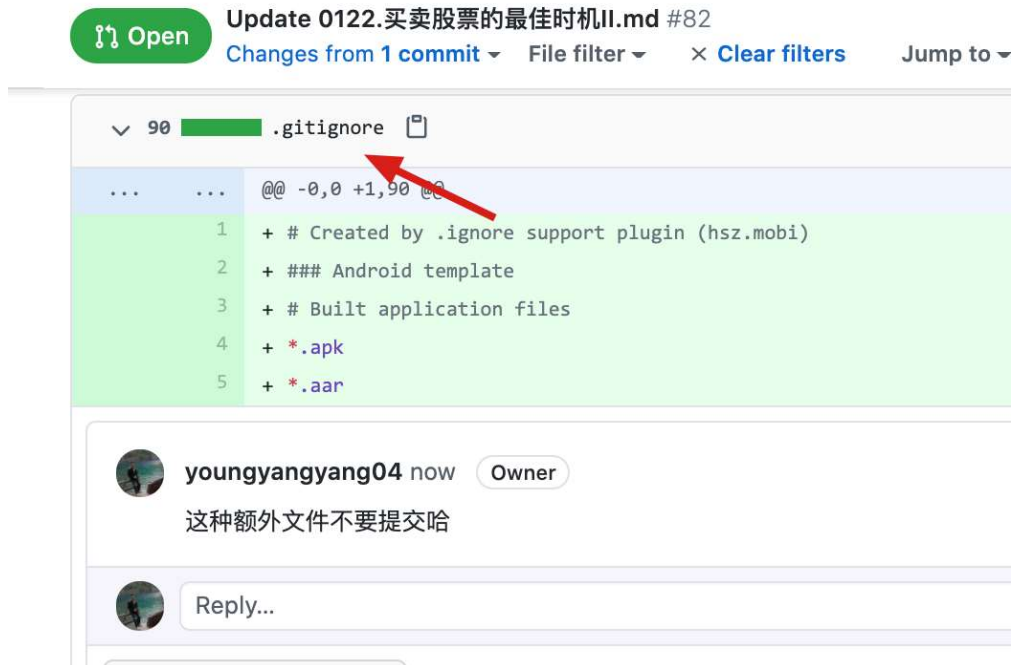
那么在处理冲突的时候 保留谁的代码，删点谁的代码呢？

我一定是看谁 代码逻辑和题解一致，代码风格好，注释友好，就保留谁的。

所以例如当你想提交Java代码的时候，即使发现该题解已经有Java版本了，只要你的代码写的好，一样可以提交，我评审合格一样可以合入代码库。

## 不要做额外修改

确保这种额外文件不要提交。



还有添加不同方法的时候，直接用正文格式写，哪种方法就可以了，不要添加目录，例如这样，这样整篇文章目录结构就有影响了。

Go:

```
+ ## 前序遍历（迭代法）：
+ ```
+ func preorderTraversal(root *TreeNode) (res []int) {
+ if root == nil {
+ return res
+ }
+ var stack []*TreeNode
```

前面不要加 `## 前序遍历（迭代法）`，直接写 `前序遍历（迭代法）` 就可以了。

当然这里也没有给代码块标记上对应的语言，应该是

``` Go Go语言代码 ```

对代码保持敬畏



- 代码运行无误
- 写的够不够简洁
- 注释清不清晰
- 备注规不规范

这也是培养大家以后协调工作的一种能力。

优化

目前 leetcode-master 中大部分题解已经补充了其他语言，但如果你发现了可以优化的地方，依然可以提交PR来优化。

甚至发现哪里有语病，也欢迎提交PR来修改，例如下面：就是把【下表】纠正为【下标】

```
3
5  * 数组的大小是受限的，而且如果元素很少，而哈希值太大会造成内存空间的浪费。
6  - * set是一个集合，里面放的元素只能是一个key，而两数之和这道题目，不仅要判断y是否存在而且还要记录y的下表位置，因为要返回x 和 y的下表，所以set 也不能用。
7  + * set是一个集合，里面放的元素只能是一个key，而两数之和这道题目，不仅要判断y是否存在而且还要记录y的下标位置，因为要返回x 和 y的下标，所以set 也不能用。
8
```

不用非要写出牛逼的代码才能提交PR，只要发现 文章中有任何问题，或者错别字，都欢迎提交PR，成为contributor。



特别注意

git add之前，要git diff 查看一下，本次提交所修改的代码是不是 自己修改的，是否 误删，或者误加的文件。